Projekt-Dokumentation: Smarter Heizungsverteiler (Keller)

Intelligente Heizkreis- und Verteilersteuerung auf Basis des **Seeed Studio XIAO ESP32-C6** mit Ansteuerung von 4 Heizkreisen (3-Wege-Mischer/Stellantriebe, Hocheffizienz-PWM-Pumpen inklusive bidirektionalem VDMA-Feedback, 1-Wire-Temperatursensorik und M-Bus-Wärmemengenzähler-Auslesung).

---

1. Systemübersicht & Architektur

- **Zentraler Mikrocontroller:** Seeed Studio XIAO ESP32-C6 (11 GPIOs)
- **I/O-Erweiterung:**
- **MCP23017 (I2C):** 16-Kanal-Digital-Expander für 12 Relais (8× Stellantriebe, 4× Pumpen 230V).
- **PCA9685 (I2C):** 16-Kanal-12-Bit-PWM-Controller für phasenstarre 1-kHz-Pumpenansteuerung.
- **Galvanische Trennung:**
- 2× 4-Kanal PC817 Optokoppler-Module (1× für PWM-Out, 1× für 75-Hz-Feedback-In).
- Getrennte Relais-Optokoppler-Versorgung (`JD-VCC`).
- **Sensorik & Bus-Systeme:**
- **M-Bus (Meter-Bus):** TTL-to-M-Bus Master-Modul zur Zählerauslesung über Hardware-UART.
- **1-Wire:** 11 bis 13× DS18B20 Temperatursensoren auf 3 getrennten Bussträngen:
  - *Strang 1 (Vorlauf):* 5× DS18B20 (4× Heizkreise Vorlauf + 1× Verteilerbalken Vorlauf).
  - *Strang 2 (Rücklauf):* 5× DS18B20 (4× Heizkreise Rücklauf + 1× Verteilerbalken Rücklauf).
  - *Strang 3 (Schaltbox):* 1 bis 3× DS18B20 (z. B. Relais- und Netzteil-Temperatur).
- **I2C-Umgebungssensorik:** 1× Kombisensor AHT (Feuchte) + BMP (Druck/Temperatur).
- **Spannungsversorgung:** Mean Well LPV-20-5 (5 V DC / 3 A / 15 W, IP67 vollvergossen).

---

2. Pinbelegung & I2C-Adressierung

XIAO ESP32-C6 (11 von 11 Pins belegt)

Pin	ESP-GPIO	Funktion / Schnittstelle	Angeschlossene Peripherie	Pegel / Besonderheit
D0	GPIO0	Pulse Capture / Timer	Pumpe 1 (EG 5 – FBH) Feedback In	75 Hz VDMA-Signal (Wilo, 3,3 V via Optokoppler)
D1	GPIO1	Pulse Capture / Timer	Pumpe 2 (OG 5 – RAD) Feedback In	75 Hz VDMA-Signal (**Cosmo**, 3,3 V via Optokoppler)
D2	GPIO2	Pulse Capture / Timer	Pumpe 3 (OG 4 – RAD) Feedback In	75 Hz VDMA-Signal (Wilo, 3,3 V via Optokoppler)
D3	GPIO21	Pulse Capture / Timer	Pumpe 4 (EG 4 – FBH) Feedback In	75 Hz VDMA-Signal (Wilo, 3,3 V via Optokoppler)
D4	GPIO22	I2C SDA	MCP23017 + PCA9685 + AHT/BMP	4,7 kΩ Pull-up auf 3,3 V
D5	GPIO23	I2C SCL	MCP23017 + PCA9685 + AHT/BMP	4,7 kΩ Pull-up auf 3,3 V
D6	GPIO19	Hardware-UART TX	M-Bus Master RXD	3,3 V TTL-Pegel
D7	GPIO20	Hardware-UART RX	M-Bus Master TXD	3,3 V TTL-Pegel
D8	GPIO9	1-Wire Bus 3	1–3× DS18B20 Strang 3 (Schaltbox: Relais / Netzteil)	Eigener 4,7 kΩ Pull-up auf 3,3 V
D9	GPIO18	1-Wire Bus 1	5× DS18B20 Strang 1: Vorlauf (4× HK + 1× Verteilerbalken)	Eigener 4,7 kΩ Pull-up auf 3,3 V
D10	GPIO3	1-Wire Bus 2	5× DS18B20 Strang 2: Rücklauf (4× HK + 1× Verteilerbalken)	Eigener 4,7 kΩ Pull-up auf 3,3 V

I2C-Busadressen (D4 = SDA, D5 = SCL)

- `0x20`: MCP23017 I/O-Expander (A0/A1/A2 auf GND)
- `0x40`: PCA9685 16-Kanal-PWM-Treiber
- `0x38`: AHT Feuchtesensor
- `0x76` / `0x77`: BMP Luftdrucksensor

3. Verdrahtungs- & Schaltungslogik

A. Spannungsversorgung (Feuchtraum / Keller)

- **Netzteil:** Mean Well LPV-20-5 (Schutzklasse II, vollvergossen nach IP67).
- **230V-Eingang:** Phase (L) und Neutralleiter (N) über 2-Leiter WAGO 221. Kein PE erforderlich.
- **5V-Sternverteilung:** Ausgang über zwei 5-Leiter WAGO 221 (1× 5V-Block, 1× GND-Block).
  - *Zweig 1 (Leistung):* Relais-Spulenversorgung (`JD-VCC`) und M-Bus-Master VCC.
  - *Zweig 2 (Logik):* XIAO ESP32-C6 (`5V`), PCA9685 und MCP23017 Logik.
- **Pufferung:** Low-ESR-Elektrolytkondensator (470 µF – 1000 µF / ≥ 10 V) direkt an den 5V/GND-Pins des ESP32 zur Glättung von Schaltspitzen.

B. Relais-Belegung & Hardware-Verriegelung (MCP23017)

Insgesamt 12 Kanäle (3× 4-Kanal-Relaismodule mit 5V-Spule):

1. 3-Wege-Stellantriebe (GPA0 bis GPA7 – Relais 1 bis 8)

Um ein gleichzeitiges Ansteuern von „AUF“ und „ZU“ (z. B. bei Softwareabsturz) physikalisch auszuschließen, ist jeder Stellantrieb über eine **Wechsler-Kaskade** hardwareverriegelt:

```
230V Phase (L) ──> [Relais 1 COM]
                        │
                        ├── [NO] ──────────────────────> Stellantrieb: AUF
                        │
                        └── [NC] ──> [Relais 2 COM]
                                          │
                                          ├── [NO] ──────> Stellantrieb: ZU
                                          │
                                          └── [NC] ──────> (unbelegt)
```

- **Mischer 1:** Relais 1 (AUF) / Relais 2 (ZU) – `GPA0` / `GPA1`
- **Mischer 2:** Relais 3 (AUF) / Relais 4 (ZU) – `GPA2` / `GPA3`
- **Mischer 3:** Relais 5 (AUF) / Relais 6 (ZU) – `GPA4` / `GPA5`
- **Mischer 4:** Relais 7 (AUF) / Relais 8 (ZU) – `GPA6` / `GPA7`

2. Pumpen-Freigabe 230V (GPB0 bis GPB3 – Relais 9 bis 12)

- **Pumpe 1 (HK 1: EG 5 – FBH):** Relais 9 – `GPB0` (Wilo-Varios PICO-STG 25/1-8)
- **Pumpe 2 (HK 2: OG 5 – RAD):** Relais 10 – `GPB1` (**COSMO CP-HY 25-75/180**)
- **Pumpe 3 (HK 3: OG 4 – RAD):** Relais 11 – `GPB2` (Wilo-Varios PICO-STG 25/1-8)

- **Pumpe 4 (HK 4: EG 4 – FBH):** Relais 12 – `GPB3` (Wilo-Varios PICO-STG 25/1-8)
- *GPB4 bis GPB7:* Reserve

C. Pumpen-Ansteuerung (PCA9685) & VDMA-Feedback mit Kennlinienfeld-Durchflussschätzung

Die 4 Hocheffizienzpumpen werden über eine Kombination aus 230V-Netzfreigabe, PWM-Sollwert und digitaler Rückmeldung gesteuert:

1. **Drehzahl-Sollwert (PCA9685 via Optokoppler):**
2. Phasenstarre **1-kHz-PWM-Ansteuerung** (0–100 % Tastverhältnis).
3. Gibt der Pumpenelektronik die Solldrehzahl bzw. Sollförderhöhe vor.
4. **VDMA-Rückmeldesignal (GPIO0, GPIO1, GPIO2, GPIO21 via Optokoppler):**
5. Die Pumpen senden ein **75-Hz-Rückmeldesignal** (VDMA-Standard) zurück an den ESP32.
6. Das Tastverhältnis (Duty Cycle) übermittelt den aktuellen Betriebszustand sowie die **aktuelle elektrische Wirkleistung / Leistungsaufnahme (P_{el} in Watt)**:
   - `0–5 %`: Signalunterbrechung / Fehler / Kein Signal
   - `5–70 %`: Aktuelle Leistungsaufnahme (linear abgebildet auf z. B. 0–45 W)
   - `75–80 %`: Standby-Zustand (0 W)
   - `80–90 %`: Warnung / Überlastbetrieb
   - `90–100 %`: Störung / Blockade (Rotor blockiert)
7. **Durchflussschätzung via Kennlinienfeld (Q_{est}):**
8. Anhand der physikalischen Pumpenkennlinie ist der resultierende Volumenstrom Q direkt abhängig von der **angeforderten Drehzahl (PWM_{soll})** und der **tatsächlich aufgenommenen Leistung (P_{el})**:
   $$Q_{\text{est}} = f(PWM_{\text{soll}}, P_{\text{el}})$$
9. In der ESP32-C6 Firmware wird hierfür ein **2D-Kennlinienfeld (Lookup-Table mit bilinearer Interpolation)** hinterlegt.
10. Daraus errechnet der ESP32 zyklisch den geschätzten Durchfluss in Litern pro Stunde (`flow_est_lh`).
11. **Nutzen:** Schnelle Durchflusserfassung im Sekundentakt, Erkennung geschlossener Heizkreisventile (Leistungsabfall bei hoher Drehzahl) und Plausibilitätsabgleich mit den M-Bus-Wärmemengenzählern.
12. **Pumpen-Referenz & Daten:** Ausführliche Dokumentation siehe `docs/wilo_varios_pico_stg.md` und digitalisierte Kennlinienpunkte in `docs/wilo_varios_pico_stg_kennlinien.csv`.

D. Virtuelle Wärmemengenzähler (Sekundengenaue Heizkreis-Leistungsberechnung)

Um nicht an das 120-Sekunden-Batteriesparintervall der M-Bus-Zähler gebunden zu sein (und um für **HK 1: EG 5**, wo der physikalische WMZ noch in Vorbereitung ist, sofort Live-Werte zu haben), berechnet der ESP32 für alle 4 Heizkreise kontinuierlich die thermodynamischen Leistungsdaten wie ein Wärmemengenzähler – **jedoch ohne kumulierte Energie (`energy_wh`)**:

1. **Eingangsgrößen je Heizkreis $i \in {1, 2, 3, 4}$:**
2. Vorlauftemperatur $T_{\text{VL}, i}$ (DS18B20 Strang 1)
3. Rücklauftemperatur $T_{\text{RL}, i}$ (DS18B20 Strang 2)

4. Geschätzter Volumenstrom $Q_{\text{est}, i}$ in Litern pro Stunde (aus Pumpenkennlinienfeld)

5. **Berechnete Werte (analog WMZ):**

6. **Spreizung (ΔT):**
   $$\Delta T_i = T_{\text{VL}, i} - T_{\text{RL}, i} \quad [\text{K}]$$
7. **Thermische Momentanleistung (P_{th} in Watt):**
   $$P_{\text{th}, i} = Q_{\text{est}, i} \cdot \Delta T_i \cdot 1{,}163\,\frac{\text{Wh}}{\text{kg}\cdot\text{K}} \quad [\text{W}]$$
   (Bedingung: Falls Pumpe AUS, $Q_{\text{est}} \le 0$ oder $\Delta T \le 0 \implies P_{\text{th}} = 0\,\text{W}$)
8. **Durchfluss (Q):** $Q_{\text{est}, i}$ in l/h
9. **Vorlauf- & Rücklauftemperatur:** $T_{\text{VL}, i}$, $T_{\text{RL}, i}$ in °C
10. **Hinweis zu kumulierter Energie:** Es wird **keine** Energie aufsummiert (`energy_wh = entfällt`), um Drift durch Schätzfehler zu vermeiden. Die verbindliche Abrechnungs- und Verbrauchsenergie stammt ausschließlich von den physikalischen M-Bus-Zählern.

---

4. M-Bus Wärmemengenzähler (Heatmeter)

> [!NOTE]
> **Portierung vom Raspberry Pi auf ESP32:**
> Das Auslesen der Wärmemengenzähler lief bisher über ein Python-Skript auf einem Raspberry Pi. Dieses Skript ist als Referenz in `heatmeter_legacy_rpi.py` gesichert und soll nun direkt in die ESP32-C6 Firmware integriert werden.

M-Bus Parameter & Schnittstelle:

- **Schnittstelle:** Hardware-UART an **D6 (TX / GPIO19)** und **D7 (RX / GPIO20)** verbunden mit dem TTL-to-M-Bus Master-Modul.
- **Baudrate:** 2400 Baud, 8 Datenbits, Even Parity, 1 Stoppbit (`SERIAL_8E1`).
- **Abfrageintervall:** 120 Sekunden (zum Batterieschutz der Zähler keine aggressiveren Retries).

Konfigurierte Wärmemengenzähler:

Zählernummer (Serial)	Zuordnung (Wohnung)	Status
`83033389`	OG5	Aktiv
`83033390`	OG4	Aktiv
`83033391`	EG4	Aktiv
`83033388`	EG5	In Vorbereitung (wird demnächst eingebaut)

Ausgelesene Register & Filterlogik:

- **Energie (Wh):** Schwelle: 500 Wh / Max-Sprung: 50.000 Wh
- **Leistung (W):** Schwelle: 50 W / Max-Sprung: 20.000 W
- **Durchfluss (l/h):** Schwelle: 1 l/h / Max-Sprung: 20.000 l/h
- **Volumen (l):** Schwelle: 10 l / Max-Sprung: 50.000 l
- **Vorlauf (°C):** Schwelle: 0,1 °C / Max-Sprung: 200,0 °C
- **Rücklauf (°C):** Schwelle: 0,1 °C / Max-Sprung: 200,0 °C
- **Spreizung (K):** Schwelle: 0,1 K / Max-Sprung: 200,0 K
- **Max-Intervall:** Spätestens nach 3600 Sekunden (1 Stunde) wird ein Wert auch ohne Überschreiten der Schwelle gesendet.

---

5. HTTP Webserver, Web-UI & REST-API Schnittstelle

Der ESP32-C6 fungiert als robuster **I/O-, Sensor- und Aktor-Controller** mit integriertem Webserver. Die **Hauptregelung** (z. B. Vorlauftemperatur-Regelung der 4 Heizkreise, Mischer-Fahrzeiten, Nachtabsenkung) läuft über HTTP-Requests von einer externen Instanz (z. B. Home Assistant, Node-RED, Python-Regel-Dienst), während die **InfluxDB-Uploads autonom auf dem ESP32 verbleiben**.

Architektur & Aufgabenverteilung:

1. **ESP32-C6 Autonom:**
2. Hardware-Schutz & Verriegelung (Wechsler-Kaskade verhindert Kurzschlüsse bei AUF/ZU).
3. Erfassung aller 18 DS18B20 Sensoren, AHT/BMP und 4× VDMA 75-Hz-Feedback.
4. Auslesen der M-Bus-Wärmemengenzähler über Hardware-UART (alle 120 s).
5. **Eigenständiger InfluxDB v2 Upload** aller Messwerte (Bucket `Heizung` / `_monitoring`).
6. Bereitstellung von Web-UI und REST-API.
7. **Externe Hauptregelung (via REST-API):**
8. Pollt zyklisch den kompletten Status über `GET /api/status`.
9. Berechnet Sollwerte und sendet Zustandsbefehle an Pumpen (`POST /api/pump`) und Mischer (`POST /api/valve`).

HTTP Basic Authentication (Sicherheit)

Sowohl die Web-UI (HTML) als auch alle REST-API-Endpunkte (JSON) sind durch **HTTP Basic Authentication** geschützt.
Die Zugangsdaten werden nicht im Git-Repository versioniert, sondern sicher im ausgelagerten `cred`-Ordner in der Datei `cred.h` konfiguriert:
* **Benutzer:** `HTTP_USER_KHV`
* **Passwort:** `HTTP_PASS_KHV`

Jegliche API-Aufrufe müssen diesen Login als Authorization-Header übergeben.

REST-API Endpunkte & JSON-Spezifikation

1. `GET /api/status` – Vollständiger Systemstatus

Liefert alle erfassten Messwerte, Aktorzustände und Systemmetriken als kompaktes JSON:


```
{
  "uptime_s": 1420,
  "wifi_rssi": -65,
  "temperatures": {
    "vorlauf": [35.2, 38.1, 32.0, 41.5],
    "ruecklauf": [28.4, 30.2, 26.5, 33.1],
    "verteilerbalken": {
      "vorlauf": 65.0,
      "ruecklauf": 35.0
    },
    "schaltbox": {
      "relais": 32.5,
      "netzteil": 38.1,
      "intern": 26.4
    },
    "ambient": {
      "temp_c": 19.8,
      "humidity_pct": 55.4,
      "pressure_hpa": 1013.2
    }
  },
  "pumps": [
    { "id": 1, "enabled": true,  "pwm": 65, "feedback_hz": 74.9, "power_w": 18.5, "status": "OK",
    { "id": 2, "enabled": false, "pwm": 0,  "feedback_hz": 0.0,  "power_w": 0.0,  "status": "STAND
    { "id": 3, "enabled": true,  "pwm": 50, "feedback_hz": 75.1, "power_w": 12.0, "status": "OK",
    { "id": 4, "enabled": false, "pwm": 0,  "feedback_hz": 0.0,  "power_w": 0.0,  "status": "STAND
  ],
  "circuits": [
    { "id": 1, "apt": "EG5", "type": "FBH", "t_fwd": 35.2, "t_bwd": 28.4, "spreading_k": 6.8, "flo
```

```
    { "id": 2, "apt": "OG5", "type": "RAD", "t_fwd": 38.1, "t_bwd": 30.2, "spreading_k": 7.9, "flo
    { "id": 3, "apt": "OG4", "type": "RAD", "t_fwd": 32.0, "t_bwd": 26.5, "spreading_k": 5.5, "flo
    { "id": 4, "apt": "EG4", "type": "FBH", "t_fwd": 41.5, "t_bwd": 33.1, "spreading_k": 8.4, "flo
  ],
  "valves": [
    { "id": 1, "state": "STOP",  "runtime_remain_ms": 0 },
    { "id": 2, "state": "CLOSE", "runtime_remain_ms": 2400 },
    { "id": 3, "state": "OPEN",  "runtime_remain_ms": 5100 },
    { "id": 4, "state": "STOP",  "runtime_remain_ms": 0 }
  ],
  "heatmeters": [
    {
      "serial": "83033389",
      "apt": "OG5",
      "status": "OK",
      "last_read_s_ago": 24,
      "energy_wh": 1245000,
      "power_w": 1250,
      "flow_lh": 210,
      "volume_l": 54230,
      "t_fwd": 38.2,
      "t_bwd": 31.5,
      "spreading_k": 6.7
    },
    {
      "serial": "83033390",
      "apt": "OG4",
      "status": "OK",
      "last_read_s_ago": 26,
      "energy_wh": 982000,
      "power_w": 980,
      "flow_lh": 180,
      "volume_l": 41820,
      "t_fwd": 37.8,
      "t_bwd": 31.9,
      "spreading_k": 5.9
    },
    {
      "serial": "83033391",
      "apt": "EG4",
      "status": "OK",
      "last_read_s_ago": 28,
      "energy_wh": 765400,
      "power_w": 0,
      "flow_lh": 0,
      "volume_l": 32110,
      "t_fwd": 22.1,
      "t_bwd": 22.0,
      "spreading_k": 0.1
    },
    {
```

```
      "serial": "83033388",
      "apt": "EG5",
      "status": "PREPARING",
      "last_read_s_ago": null,
      "energy_wh": 0,
      "power_w": 0,
      "flow_lh": 0,
      "volume_l": 0,
      "t_fwd": null,
      "t_bwd": null,
      "spreading_k": null
    }
  ],
  "influx": {
    "connected": true,
    "last_write_s_ago": 8
  }
}
```

2. `POST /api/pump` – Pumpe ansteuern

Schaltet das 230V-Freigaberelais (MCP23017) und setzt das 1-kHz-PWM-Signal (PCA9685, 0–100 %):

```
{
  "id": 1,
  "enabled": true,
  "pwm": 70
}
```

Antwort: `{"success": true, "pump": 1, "enabled": true, "pwm": 70}`

3. `POST /api/valve` – 3-Wege-Mischer verfahren

Aktiviert für eine definierte Zeitspanne die Fahrtrichtung (hardware- und softwareverriegelt):

```
{
  "id": 1,
  "action": "open",       // "open" | "close" | "stop"
  "duration_ms": 4000     // Laufzeit des Impulses in Millisekunden
}
```

- Wenn `duration_ms` abgelaufen ist, schaltet der ESP32 das Relais automatisch wieder stromlos (`STOP`).
- Ein Aufruf mit `"action": "stop"` stoppt die Bewegung sofort.

4. `POST /api/cmd` – Kombinierter Regelungs-Batch

Erlaubt der Hauptregelung die Aktualisierung aller Kreise in einem einzigen HTTP-Request:

```
{
  "pumps": [
    { "id": 1, "enabled": true, "pwm": 60 },
    { "id": 2, "enabled": false, "pwm": 0 }
  ],
  "valves": [
    { "id": 1, "action": "open", "duration_ms": 3000 },
    { "id": 2, "action": "stop" }
  ]
}
```

5.2 Web-UI Dashboard (GET /) – Visueller Aufbau & Live-Monitoring

Das Web-Dashboard wird als kompakte Single-Page-Application (SPA in modernem Dark Mode) direkt aus dem Flash-Speicher des ESP32-C6 serviert. Es dient der **Echtzeit-Überwachung**, der **manuellen Handebene (Service)** und dem **Notfallbetrieb**.

Layout & Visuelle Struktur:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│  🔥 SMARTER HEIZUNGSVERTEILER (KHV) - DASHBOARD              [NOTFALL-MODUS] │
│  WLAN: Verbunden (-65 dBm) │ Uptime: 14h 22m │ Modus: WINTER │ InfluxDB: OK │
├──────────────────────────────────────────────────────────────────────────────┤
│  ═══ VERTEILERBALKEN (HAUPTVERSORGUNG) ═══════════════════════════════════ │
│  Vorlauf: 65.2 °C    │    Rücklauf: 35.0 °C    │    Spreizung (ΔT): 30.2 K    │
├──────────────────────────────────────────────────────────────────────────────┤
│  ═══ DIE 4 HEIZKREISE (LIVE-ÜBERWACHUNG & AKTORIK) ═══════════════════════ │
│  ┌──────────────────┐┌──────────────────┐┌──────────────────┐┌──────────────┐│
│  │ HK 1: FBH (EG 5) ││ HK 2: RAD (OG 5) ││ HK 3: RAD (OG 4) ││ HK 4: FBH (EG 4) │
│  │ Typ: Fußboden    ││ Typ: Fußboden    ││ Typ: Heizkörper  ││ Typ: Heizk. │
│  │ VL: 38.2 °C [MAX]││ VL: 37.5 °C      ││ VL: 58.1 °C      ││ VL: 55.4 °C │
│  │ RL: 30.1 °C      ││ RL: 29.8 °C      ││ RL: 42.0 °C      ││ RL: 39.5 °C │
│  │ Mischer: STOP    ││ Mischer: AUF 3s  ││ Mischer: STOP    ││ Mischer: ZU │
│  │ Pumpe: EIN (65%) ││ Pumpe: EIN (50%) ││ Pumpe: EIN (70%) ││ Pumpe: AUS │
│  │ 18.5W · 215 l/h  ││ 12.0W · 160 l/h  ││ 22.0W · 240 l/h  ││ 0W · 0 l/h    │
│  │ [Hand] [Auf] [Zu]││ [Hand] [Auf] [Zu]││ [Hand] [Auf] [Zu]││ [Hand] ...  │
│  └──────────────────┘└──────────────────┘└──────────────────┘└──────────────┘│
├──────────────────────────────────────────────────────────────────────────────┤
│  ═══ WÄRMEMENGENZÄHLER (M-BUS LIVE-DATEN) ═══════════════════════════════ │
│  ┌────────────────┐ ┌────────────────┐ ┌────────────────┐ ┌────────────────┐│
│  │ WMZ 1: EG 5    │ │ WMZ 2: OG 5    │ │ WMZ 3: OG 4    │ │ WMZ 4: EG 4    │
│  │ Leistung: 1.2kW│ │ (In Vorbereit.)│ │ Leistung: 980 W│ │ Leistung: 1.4kW│
```

```
│  │ Fluss: 190 l/h │ │ Status: WARTEND│ │ Fluss: 160 l/h │ │ Fluss: 220 l/h ||
│  │ Energie: 765kWh│ │ Energie: --    │ │ Energie: 982kWh│ │ Energie:1.24MWh||
│  │ VL/RL: 38 / 30 │ │ VL/RL: -- / -- │ │ VL/RL: 58 / 42 │ │ VL/RL: 55 / 39 ||
│  └────────────────┘ └────────────────┘ └────────────────┘ └────────────────┘|
├──────────────────────────────────────────────────────────────────────────────┤
│  ═══ SCHALTSCHRANK-SICHERHEIT & KLIMA ═══════════════════════════════════ │
│  Relais: 32.5 °C (OK) │ Netzteil: 38.1 °C (OK) │ Keller: 19.8 °C / 55% rH    │
└──────────────────────────────────────────────────────────────────────────────┘
```

5.3 Autonomer Notfallmodus & Ausfall-Sicherheit (WLAN / RPi-Watchdog)

Fällt die übergeordnete Steuerung (Raspberry Pi / Home Assistant / Node-RED) oder das Netzwerk aus, darf das Gebäude im Winter keinesfalls auskühlen. Der ESP32-C6 besitzt daher eine autarke **Notfall-Betriebslogik**:

1. Auslösekriterien für den Notfallmodus:

- **Manuell:** Über den Notfall-Button im Web-Dashboard (z. B. für Schornsteinfeger-Prüfung, Wartung oder manuelle Volllast).
- **Automatischer RPi-Watchdog:** Wenn die externe Hauptregelung über die REST-API länger als **15 Minuten** keine Steuerbefehle (`POST /api/...`) an den ESP32 gesendet hat.
- **WLAN-Verbindungsverlust:** Wenn der ESP32 länger als **15 Minuten** die Verbindung zum WLAN-Router verliert.

2. Regelverhalten im Notfallmodus:

- **Pumpen 1–4:** Werden eingeschaltet (Freigaberelais aktiv, Grunddrehzahl ca. **60–70 % PWM**).
- **Radiatoren-Heizkreise (HK 2: OG 5 & HK 3: OG 4):**
- Da Heizkörper unkritisch gegen hohe Vorlauftemperaturen sind, fahren die Mischer auf **80–100 % AUF** (Balkentemperatur bis 65 °C), um den ungedämmten Altbau maximal zu heizen.
- **Fußbodenheizung (HK 1: EG 5 & HK 4: EG 4):**
- Werden durch einen **autarken Dreipunkt-Schrittregler mit Sensor-Fusion und Gradienten-Bremse** auf maximaler sicherer Temperatur gehalten (siehe 5.4).
- Sobald wieder gültige REST-Befehle der Hauptregelung eintreffen oder der Notfallmodus im UI deaktiviert wird, schaltet das System nahtlos zurück in den regulären Regelbetrieb.

5.4 FBH-Notfallregelung & Übertemperaturschutz (Altbau)

Das Gebäude besitzt zwei grundverschiedene Heizkreis-Arten:
* **Heizkreis 1 (EG 5) & Heizkreis 4 (EG 4): Fußbodenheizung (FBH)**
* **Heizkreis 2 (OG 5) & Heizkreis 3 (OG 4): Heizkörper (Radiatoren)**

> [!CAUTION]
> **FBH-Schutz hat immer höchste Priorität:**
> Eine Fußbodenheizung darf niemals die Vorlauftemperatur von Heizkörpern oder des Verteilerbalkens (bis 70 °C) abbekommen. Hohe Temperaturen führen zu Zerstörung des Estrichs, Lösen von Fliesen und irreversiblen Schäden an Parkett- und Bodenbelägen!

Physikalische Herausforderung bei Anlegefühlern (DS18B20 vs. WMZ):

1. **Große thermische Totzeit (30–60 Sekunden):** Die DS18B20 sind außen am Rohr befestigt. Bis heißes Wasser aus dem Mischer die Rohrwand und den Fühlerkörper durchwärmt, vergeht bis zu eine Minute (PT1/PT2-Verhalten).
2. **Kühlkörper-Effekt (Nie-Echtwert-Erreichung):** Die Sensorrückseite gibt Wärme an die kühlere Kellerluft ab. Ohne Korrektur misst der Rohranleger $2 \dots 3\,\text{K}$ kälter als das Wasser im Rohrinneren (z. B. Rohrwand 42 °C, Kernwasser bereits 45 °C!).
3. **Asynchrone Sensorik:** Die extrem genauen WMZ-Tauchhülsen senden nur alle **120 Sekunden** (Batterieschutz), an HK 1 ist der WMZ noch in Vorbereitung.
4. **Warum Standard-PID-Regler hier versagen würden (Hunting-Gefahr):**
5. Ein normaler PI/PID-Regler integriert den Regelfehler permanent auf (I-Anteil).
6. Da der Anlegefühler erst mit 30–60 s Verzug reagiert, denkt ein Standard-PID, es sei immer noch zu kalt, und fährt den Mischer viel zu weit auf.
7. Wenn die Hitzewelle nach einer Minute am Sensor ankommt, schießt die Temperatur unweigerlich über 50 °C hinaus und löst eine Notabschaltung aus (*Hunting / Aufschaukeln*).

Lösung: 3-stufige Sensor-Fusion & Notfall-Regelungsarchitektur:

```
Alle 2-5s: DS18B20 (T_DS) ────────┐
                                  ▼
                        [ T_eff = T_DS + Offset ] ──► [ Gradienten-Bremse (dT/dt) ]
                                  ▲                              │
Alle 120s: WMZ Tauchhülse (T_WMZ) ┘                              ▼
           Offset = T_WMZ - T_DS (langsam gleitend)      [ Asymmetrischer 3-Punkt-Schrittregler ]
                                                                 │
                                                        (45-60s Totzeit-Pause nach Impuls)
```

1. **Sensor-Fusion & Offset-Lernen (Observer-Kompensation):**
2. Bei aktivem WMZ berechnet der ESP32 alle 120 s den Mess-Offset: $\Delta_{\text{offset}} = T_{\text{WMZ}} - T_{\text{DS18B20}}$.
3. Der Regler nutzt sekündlich die fusionierte Temperatur: $T_{\text{eff}} = T_{\text{DS18B20}} + \Delta_{\text{offset}}$.

4. **Reiner DS18B20-Betrieb (z. B. HK 1 ohne WMZ):** Liegt kein WMZ vor, wird $\Delta_{\text{offset}} = 0$ gesetzt und der Regler-Sollwert sicherheitshalber auf **max. 41–42 °C** begrenzt (entspricht ca. 44–45 °C Kernwasser).

5. **Gradienten-Bremse (D-Anteil / Trend-Erkennung dT/dt):**
6. Steigt die Vorlauftemperatur schneller als mit **$+0{,}5\,\text{K} \text{ pro } 10\,\text{Sekunden}$**, werden alle weiteren AUF-Impulse **sofort verriegelt**.
7. Der Regler wartet ab, bis die Hitzewelle am Sensor vollständig durchgeschlagen ist, bevor erneut geregelt wird.
8. **Erzwungene Totzeit-Pause (45 bis 60 Sekunden Einschwingzeit):**
9. Nach jedem Stellimpuls (z. B. 2 Sekunden AUF) hält der Regler **mindestens 45–60 Sekunden die Füße still**, damit die Rohrwand das thermische Gleichgewicht erreicht.
10. Es gibt **keinen I-Anteil**, der weglaufen könnte. Stellentscheidungen fallen ausschließlich im thermisch eingeschwungenen Zustand ($\frac{dT}{dt} \approx 0$).
11. **Asymmetrischer Dreipunkt-Schrittregler (Sollwert: $44{,}0\,^\circ\text{C}$):**

Vorlauftemperatur (T_{eff})	Mischer-Aktion	Impuls / Pause	Regelverhalten
$< 42{,}0\,^\circ\text{C}$	**AUF**-Impuls	$t = (44 - T) \cdot 0{,}8\,\text{s}$ (max. 3 s) / **45–60 s Pause**	Sanftes Öffnen ohne Überschwingen
$42{,}0 \dots 45{,}0\,^\circ\text{C}$	**STOP (Totzone)**	Relais stromlos	**Kein Verschleiß, maximale Wärme gehalten**
$45{,}1 \dots 47{,}9\,^\circ\text{C}$	**ZU**-Impuls	$t = (T - 44) \cdot 1{,}5\,\text{s}$ (3–5 s) / **15 s Pause**	**Asymmetrisch:** Bremst 3x schneller ab als er öffnet
$48{,}0 \dots 49{,}9\,^\circ\text{C}$	**Dauerhaft ZU**	Mischer fährt kontinuierlich ZU	Sofortiger Schutz vor 50 °C
$\ge 50{,}0\,^\circ\text{C}$	**Hard-Cutoff**	Mischer voll ZU + **Pumpe AUS** + Alarm	Physikalischer Schutz & Telegram-Push

Mechanische Montageempfehlung:

- Zwischen Rohr und DS18B20-Sensor unbedingt **Wärmeleitpaste** anbringen.
- Den Sensor mit **Rohrisolierung (Armaflex/Schaumstoff)** nach außen dämmen, um den Kühleffekt der Kellerluft zu minimieren.

Autonome Sicherheitsabschaltung bei Übertemperatur (Hard-Cutoff):

Steigt die Vorlauftemperatur an HK 1 oder HK 4 trotz aller Regelung über **50,0 °C**:
1. Der ESP32 fährt den Mischer **sofort und ohne Verzögerung voll auf ZU** (Vorrang vor allen externen API-Befehlen und vor dem Notfallmodus).
2. Steigt die Temperatur trotz ZU-Fahrt weiter an (z. B. Mischer defekt oder undicht), schaltet der ESP32 das **230V-Pumpenrelais des betroffenen FBH-Kreises ab**, um den Heißwasserzufluss physikalisch zu stoppen.
3. Über die Telegram-Bot-API wird sofort eine **Warnmeldung mit Alarmton** abgesetzt:
"🚨 ALARM: FBH Übertemperatur an HK 1 (51.2 °C)! Mischer ZU gefahren & Pumpe abgeschaltet!"

5.5 Sommer- / Winter-Umschaltung (Heizgrenze für Altbau)

Aufgrund der thermischen Trägheit und der höheren Transmissionswärmeverluste des Altbaus muss die Heizung im Frühjahr länger laufen und im Herbst früher anspringen als in modernen Gebäuden:

1. Heizgrenztemperatur: **17,0 °C**

- **Winterbetrieb (Heizbetrieb):** Aktiv, solange der gleitende 24-Stunden-Mittelwert der Außentemperatur **unter 17,0 °C** liegt.
- **Sommerbetrieb (Heizung AUS):** Wird erst aktiviert, wenn es dauerhaft warm ist, d. h. der gleitende Tagesmittelwert **über 17,0 °C** ansteigt (oder manuell über das Web-UI).

2. Typische Betriebsperioden (Standort Deutschland / Mitteleuropa im Altbau):

- **Heizperiode (Winterbetrieb):** Ca. **Mitte September bis Ende Mai** (ca. 8,5 Monate im Jahr).
- **Sommerpause:** Ca. **Juni bis Anfang September** (ca. 3,5 Monate).

3. Verhalten im Sommerbetrieb:

- **Energieeinsparung:** Alle 4 Heizkreispumpen werden stromlos geschaltet (Relais AUS, 0 % PWM). Alle 4 Mischer fahren voll auf ZU.
- **Antiblockierschutz (Wöchentlicher Kesselschutzlauf):**
  Um ein Festsetzen („Festbacken") von Pumpenwellen und Mischer-Dichtungen über die Sommermonate zu verhindern, führt der ESP32 **jeden Sonntag um 11:00 Uhr** einen automatischen 60-Sekunden-Schutzlauf durch:
- Jede Pumpe läuft für 60 Sekunden auf 50 % Drehzahl an.
- Jeder 3-Wege-Mischer fährt für 30 Sekunden AUF und anschließend wieder voll ZU.

---

6. Smartes InfluxDB Sende- & Filtersystem

Der ESP32-C6 besitzt ein intelligentes Datenmanagement für InfluxDB v2, um Datenbankmüll und Fehlmessungen (z. B. 1-Wire Glitches, 85°C-Power-On-Resets der DS18B20) zuverlässig auszufiltern und den Netzwerkverkehr zu minimieren.

```
[ Sensor-Messwert ]
        │
        ▼
[ 1. Plausibilitäts- & Ausreißerfilter ] ──> Glitch erkannt? (z.B. 30°C -> 90°C -> 30°C)
        │                                     └──> Vorübergehend halten; erst bei Bestätigung durc
        ▼ (Gültiger Messwert)
[ 2. Delta- & Heartbeat-Prüfung ] ──────────> Hat sich der Wert um >= Threshold geändert?
        │                                     ODER ist Max-Intervall abgelaufen?
        ▼ (Senden erforderlich)                └──> NEIN: Wert verwerfen / nicht senden
[ 3. 60s Batch-Sammlung ]
        │
        ▼ (Alle 60 Sekunden)
[ 4. Gebündelter InfluxDB HTTP-Post ]
```

A. 2-Schritt-Ausreißerfilter (Plausibilitätsprüfung)

Typische Messfehler bei 1-Wire (z. B. Bitfehler oder der 85°C-Resetwert) zeichnen sich durch extreme, einmalige Nadelimpulse aus (z. B. `30°C → 90°C → 30°C`).

- **Funktionsweise:**
- Weicht ein neuer Messwert um mehr als `max_jump` (z. B. 15 K) vom letzten bestätigten Wert ab, wird er **nicht sofort übernommen**, sondern als *Verdachtskandidat* zwischengespeichert.
- Erst wenn der **nächste Messwert** diesen Trend bestätigt (d. h. nahe am Kandidaten bleibt), wird der Sprung als realer Temperaturanstieg (z. B. Warmwasserzirkulation startet) akzeptiert.
- Fällt der nächste Messwert hingegen wieder in den ursprünglichen Bereich zurück, wird der Ausreißer lautlos verworfen und ein Fehlmessungs-Zähler inkrementiert.

B. Minütliches Batching (Zyklus: 60 Sekunden)

- Messwerte werden kontinuierlich erfasst (z. B. DS18B20 alle 5–10 s, AHT/BMP alle 10 s, Pumpen kontinuierlich).
- Zu sendende Datenpunkte werden im RAM gesammelt.
- **Genau einmal pro Minute (alle 60 s)** werden alle gesammelten Punkte in einem einzigen gebündelten HTTP-POST-Request an InfluxDB übertragen.
- **Vorteile:** Minimale WLAN-Sendezeit, geringe CPU-Last und keine Fragmentierung auf dem InfluxDB-Server.

C. 3-Faktor-Sendeentscheidung (Mindestabstand, Schwellwert & Heartbeat)

Um eine Datenflut in InfluxDB zuverlässig zu unterbinden (z. B. bei Sensorrauschen oder raschen Pendelungen), wird ein Datenpunkt nur dann in den minütlichen Sende-Batch aufgenommen, wenn folgende Kriterien erfüllt sind:

1. **Mindestabstand eingehalten (Rate Limit / `min_interval`):**
2. Seit dem letzten Senden dieses spezifischen Messwerts muss die konfigurierte Mindestzeit vergangen sein:
   $$\Delta t_{\text{seit_letztem_Senden}} \ge \text{min_interval}$$
3. Ist diese Zeit noch nicht verstrichen, wird der Wert verworfen (nicht gesendet), selbst wenn der Schwellwert überschritten wäre.
4. **Wertänderung ODER Maximalzeit (Heartbeat / `max_interval`):**
5. Ist der Mindestabstand erfüllt, wird gesendet, sobald mindestens eine Bedingung zutrifft:
   - **Schwellwert überschritten:** $|\text{Aktueller_Wert} - \text{Letzter_Gesendeter_Wert}| \ge \text{Threshold}$
   - **Maximalzeit abgelaufen:** $\Delta t_{\text{seit_letztem_Senden}} \ge \text{max_interval}$ (standardmäßig 600 s / 10 min als Lebenszeichen).

```
Senden an InfluxDB? = (Δt >= min_interval) AND (|ΔWert| >= Threshold OR Δt >= max_interval)
```

Konfiguration der einzelnen Messgrößen & Filter:

Kategorie / Sensor	Messgröße / Register	Threshold (Δ Min)	Max-Sprung (`max_jump`)	Mindestabstand (`min_interval`)	Einheit
Heizkreise 1–4 (1-Wire)	Vorlauftemperatur	`0,2`	`15,0`	`60 s`	°C
Heizkreise 1–4 (1-Wire)	Rücklauftemperatur	`0,2`	`15,0`	`60 s`	°C
Verteilerbalken (1-Wire)	Vorlauftemperatur	`0,3`	`20,0`	`60 s`	°C
Verteilerbalken (1-Wire)	Rücklauftemperatur	`0,3`	`20,0`	`60 s`	°C
Schaltbox-Sicherheit (1-Wire)	Relais-Kaskade Temperatur	`0,5`	`20,0`	`60 s`	°C
Schaltbox-Sicherheit (1-Wire)	Mean-Well-Netzteil Temperatur	`0,5`	`20,0`	`60 s`	°C
Schaltbox-Sicherheit (1-Wire)	Gehäuse-Innentemperatur	`0,5`	`20,0`	`60 s`	°C
Umgebungsklima (I2C)	AHT20 Raumtemperatur	`0,2`	`10,0`	`120 s`	°C
Umgebungsklima (I2C)	AHT20 Luftfeuchtigkeit	`1,0`	`20,0`	`120 s`	% rH
Umgebungsklima (I2C)	BMP280 Luftdruck	`0,5`	`20,0`	`120 s`	hPa
Pumpen 1–4 (VDMA Feedback)	Leistungsaufnahme (`power_w`)	`0,5`	`20,0`	`60 s`	W
Pumpen 1–4 (VDMA Feedback)	Durchflussschätzung (`flow_est_lh`)	`5,0`	`200,0`	`60 s`	l/h
Pumpen 1–4 (VDMA Feedback)	Feedback-Signalfrequenz	`1,0`	`40,0`	`60 s`	Hz
Virtuelle WMZ 1–4 (Pumpe & DS18B20)	Thermische Leistung (`heat_power_w`)	`50`	`20.000`	`60 s`	W
Virtuelle WMZ 1–4 (Pumpe & DS18B20)	Spreizung (`spreading_k`)	`0,2`	`20,0`	`60 s`	K

Kategorie / Sensor	Messgröße / Register	Threshold (Δ Min)	Max-Sprung (`max_jump`)	Mindestabstand (`min_interval`)	Einheit
Wärmemengenzähler (M-Bus)	Energie (`energy_wh`)	`500`	`50.000`	`120 s`	Wh
Wärmemengenzähler (M-Bus)	Momentanleistung (`power_w`)	`50`	`20.000`	`120 s`	W
Wärmemengenzähler (M-Bus)	Volumenstrom (`flow_lh`)	`1`	`20.000`	`120 s`	l/h
Wärmemengenzähler (M-Bus)	Kumuliertes Volumen (`volume_l`)	`10`	`50.000`	`120 s`	l
Wärmemengenzähler (M-Bus)	Vorlauftemperatur (`t_fwd`)	`0,1`	`20,0`	`120 s`	°C
Wärmemengenzähler (M-Bus)	Rücklauftemperatur (`t_bwd`)	`0,1`	`20,0`	`120 s`	°C
Wärmemengenzähler (M-Bus)	Spreizung (`spreading_k`)	`0,1`	`20,0`	`120 s`	K

---

7. Alarmierung & Monitoring (Zweistufiges Fehler-Konzept)

Zur Ausfallsicherheit und Diagnose wird strikt zwischen **dringenden/kritischen Alarmen** und **nicht-dringenden Diagnose-Ereignissen** unterschieden:

```
                Fehler aufgetreten
                        |
          ┌─────────────┴──────────────┐
          ▼                            ▼
 [ Dringend / Kritisch ]    [ Nicht dringend / Diagnose ]
          |                            |
          ▼                            ▼
   Telegram-Alarm            InfluxDB Bucket: `_monitoring`
  (Push aufs Handy)          (Measurement: `service_log`)
  - Pumpe Blockade/Fehler    - Plausibilitätsfilter-Spikes
  - Überhitzung Schaltbox    - Vereinzelte 1-Wire CRC-Fehler
  - Zähler Watchdog (>60m)   - Einzelne M-Bus Timeouts
  - Kompletter Busausfall    - Ungültige API-Requests
```

A. Telegram-Alarmierung (Kritische Fehler via Bot API)

Wichtige Zustände, die ein Eingreifen erfordern oder auf Hardware-Defekte hinweisen, werden direkt über den Telegram-Bot aufs Smartphone gesendet (Zugangsdaten `BOT_TOKEN` und `CHAT_ID` aus `cred.h`).

- **Alarme & Auslösekriterien:**
- **Pumpenfehler (VDMA-Feedback):** Eine Pumpe signalisiert über das 75-Hz-Signal einen Störzustand (z. B. Rotor blockiert, Trockenlauf oder Drehzahl-Sollwert stark verfehlt).
- **Überhitzung der Schaltbox:** Ein Sensor an Relais oder Netzteil überschreitet eine Warnschwelle (z. B. `> 60 °C`).
- **Wärmemengenzähler-Watchdog:** Wie im früheren RPi-Dienst wird ein Alarm ausgelöst, wenn seit mehr als **60 Minuten** keine validen Zählerdaten mehr empfangen wurden. Sobald wieder Daten eintreffen, folgt eine Entwarnungs-Nachricht.
- **Kompletter Bus-Ausfall:** Strang 1, 2 oder 3 liefert bei einem Abfragezyklus 0 antwortende DS18B20-Sensoren.
- **Dauerhafter Netzausfall:** InfluxDB-Server oder lokales WLAN seit mehr als 15 Minuten nicht erreichbar.
- **Reboot-Meldung:** Info beim Geräteneustart inklusive Neustartgrund (z. B. Power-On, Software-Reset, Watchdog).
- **Flutschutz:** Jeder Alarmtyp besitzt eine Sperrzeit (Cooldown), damit bei anhaltenden Fehlern nicht das Postfach geflutet wird.

B. InfluxDB Monitoring (`_monitoring`-Bucket für Service-Logs)

Nicht-dringende Diagnosemeldungen, Glitches und temporäre Störungen belasten nicht das Smartphone, sondern werden in das InfluxDB-Diagnosebucket `_monitoring` geschrieben.

- **Bucket:** `_monitoring` (wie im RPi-Dienst)
- **Measurement:** `service_log`
- **Schema:**
- **Tags:**
  - `level`: `"info"`, `"warning"`, `"error"`, `"critical"`
  - `component`: `"1wire"`, `"mbus"`, `"i2c"`, `"pump"`, `"influx"`, `"api"`, `"system"`
  - `channel`: z. B. `"pumpe_1"`, `"hk2_vorlauf"`, `"meter_83033389"`
- **Fields:**
  - `message`: Klartext-Fehlerbeschreibung (z. B. `"Unlogic jump from 32.1°C to 89.4°C discarded"`).
  - `event_count`: Ganzzahl `1` (erlaubt einfache Aggregation und Häufigkeitsanalysen in Grafana).
  - `raw_value`: Verworfenes Messergebnis (optional).
- **Typische geloggte Ereignisse:**
- Verwerfen eines Ausreißers durch den 2-Schritt-Plausibilitätsfilter (z. B. 90°C-Spike).

- Vereinzelte Checksummen- (CRC-) Fehler beim Auslesen eines DS18B20-Sensors.
- Einzelner Timeout bei einem M-Bus-Telegramm vor erfolgreichem Retry.
- Wiederverbindung nach kurzem WiFi-Reconnect.
- Abgewiesene, syntaktisch falsche REST-API JSON-Befehle.

---

8. PlatformIO & Workspace-Struktur

📁 Dateistruktur

```
khv/
├── .gitignore
├── khv.code-workspace         <-- Multi-Root Workspace (enthält KHV & ../cred)
├── platformio.ini             <-- XIAO ESP32-C6 Konfiguration & -I"../cred"
├── heatmeter_legacy_rpi.py    <-- Referenzcode M-Bus Zählerauslesung (RPi)
├── README.md
├── include/                   <-- Header-Dateien
├── lib/                       <-- Lokale Bibliotheken
├── src/
│   └── main.cpp               <-- Hauptprogramm
└── test/                      <-- Unit-Tests
```

🔑 Zentrale Zugangsdaten (`cred`)

- Globaler Speicherort: `../cred/cred.h`
- Enthält WLAN, InfluxDB v2 Zugangsdaten und Telegram Bot-Token (`BOT_TOKEN`, `CHAT_ID`).
- In `platformio.ini` eingebunden via:
  `ini build_flags = -I"../cred"`
- Direkter Import in C++ über `#include <cred.h>`.

🚀 Workspace öffnen

1. In VS Code: **Datei -> Arbeitsbereich aus Datei öffnen...** (*File -> Open Workspace from File...*)
2. khv.code-workspace öffnen, um sowohl das Projekt als auch den `cred`-Ordner parallel im Explorer zu verwalten.

🐙 Versionsverwaltung (Git & GitHub)

- **GitHub Repository:** Private Repository unter `https://github.com/FriedrichNowacki/khv`
- **Branch:** `main`
- **Sicherheit:** Der Ordner `../cred` liegt außerhalb des Git-Baums – Passwörter und Tokens gelangen niemals in das Repository.

- **Nützliche Befehle:**
- Neuen Stand sichern & pushen:

  `bash git add . git commit -m "Beschreibung der Aenderung" git push`
- Ungespeicherte Änderungen verwerfen: `git restore .`
- Auf den gesicherten GitHub-Stand zurücksetzen: `git reset --hard origin/main`